Clasificación de sentimientos en reseñas de películas de IMDb con una red neuronal recurrente LSTM

Juan Sebastian Hernandez, Juan Carlos Echeverri, Juan Fernando Contreras

1. Introducción

Las reseñas de películas en IMDb son texto libre escrito por usuarios, y su polaridad (positiva o negativa) sirve para resumir la opinión del público o para alimentar sistemas de recomendación. La tarea es difícil porque el sentimiento depende del orden y del contexto: una negación, una ironía o una queja al final pueden cambiar el sentido de un texto lleno de elogios.

Los primeros clasificadores de sentimiento representaban cada reseña como una bolsa de palabras. Con esa representación y una SVM, Pang *et al.* [1] clasificaron 1400 reseñas de cine con 82,9 % de exactitud, aunque así se pierde el orden de las palabras. Maas *et al.* [2] publicaron un corpus de 50 000 reseñas de IMDb y, al combinar vectores de palabras aprendidos con la bolsa de palabras, llegaron a 88,89 %. Las redes recurrentes (RNN) leen la reseña token a token y resumen lo leído en un estado oculto. En una RNN simple el gradiente se desvanece en secuencias largas [3]; la LSTM [4] lo mitiga con compuertas que deciden qué información se conserva y cuál se olvida en cada paso.

El objetivo de este trabajo es clasificar reseñas de IMDb como positivas o negativas con una red recurrente. Para ello construimos un pipeline que limpia y tokeniza el texto, arma el vocabulario solo con datos de entrenamiento y convierte cada reseña en una secuencia de índices. Con esas secuencias entrenamos una LSTM con una capa de embedding aprendida desde cero, la comparamos con una GRU usando tres semillas por arquitectura y evaluamos en un conjunto de prueba aislado con *precision*, *recall* y F1 por clase.

2. Metodología

Usamos el archivo movie.csv del conjunto *IMDB Movie Ratings Sentiment Analysis* de Kaggle [5]: 40 000 reseñas en inglés con el texto y una etiqueta binaria, 1 para las positivas (19 981) y 0 para las negativas (20 019). Encontramos 277 textos repetidos de forma exacta, ninguno con etiquetas distintas, y dejamos una sola copia de cada uno (39 723 reseñas) para que la misma reseña no quede en entrenamiento y en prueba. La división es estratificada, con semilla 42: 70 % para entrenamiento (27 806), 15 % para validación (5958) y 15 % para prueba (5959).

Para tokenizar decodificamos las entidades HTML, cambiamos etiquetas como
 por espacios y pasamos el texto a minúsculas. Una expresión regular conserva palabras, números, contracciones como *didn't* y los signos ! y ?, que suelen acompañar opiniones fuertes. El vocabulario tiene 20 000 entradas: los 19 998 tokens más frecuentes en entrenamiento, <PAD> para el relleno y <UNK> para las palabras fuera del vocabulario. No truncamos las reseñas. El DataLoader rellena cada lote hasta la longitud de su reseña más larga y entrega también la longitud real de cada una.

El modelo tiene tres capas. La primera es un embedding que asigna a cada índice un vector entrenable de 64 dimensiones. El enunciado menciona Word2Vec o GloVe [6]; preferimos aprender los vectores junto con el clasificador para que se ajusten a la tarea de sentimiento y no depender de un vocabulario externo. La segunda es una LSTM unidireccional de una capa con estado oculto de 64 unidades, elegida porque retiene contexto en reseñas de cientos de tokens [4]. Tomamos su salida en el último token real de cada reseña, de modo que el relleno no influye en la predicción. La tercera es una capa densa de 64 a 1 que produce un logit, y su sigmoide es la probabilidad de la clase positiva. El modelo suma 1 313 345 parámetros, el 97,5 % de ellos en el embedding.

Entrenamos con Adam [7], tasa de aprendizaje de 10^{-3} y lotes de 32 reseñas. La pérdida es BCEWithLogitsLoss de PyTorch, que integra la sigmoide en la entropía cruzada binaria y es numéricamente más estable que aplicarlas por separado. Permitimos hasta 5 épocas con parada temprana: si la pérdida de validación no mejora en 2 épocas seguidas, el entrenamiento se detiene, y al final se restauran los pesos de la época con menor pérdida de validación. Todo corrió en CPU con PyTorch 2.14.

Para comparar arquitecturas y medir la variabilidad repetimos el entrenamiento con las semillas 0, 1 y 2 para la LSTM y para una GRU [8] con el mismo embedding y el mismo estado oculto. La GRU tiene dos compuertas en lugar de tres y no guarda una celda de memoria aparte, así que su capa recurrente tiene 24 960 parámetros frente a 33 280 en la LSTM. La semilla cambia la inicialización y el orden de los lotes, no la división de los datos. El análisis de errores usa la primera corrida de la LSTM, con semilla 42.

En prueba, una reseña se clasifica como positiva si su probabilidad es al menos 0,5. Reportamos *precision*, *recall* y F1 por clase, su promedio macro y la exactitud, y para las semillas el promedio ± la desviación estándar muestral. Las clases están casi balanceadas, así que la exactitud es informativa; *precision* y *recall* por clase muestran además hacia qué lado se equivoca el modelo, y el F1 macro da el

mismo peso a las dos polaridades.

3. Resultados

La Tabla 1 compara la corrida principal con las tres semillas de cada arquitectura. La LSTM con semilla 42 obtiene 86,64 % de exactitud y F1 macro de 0,8662. Con las semillas 0, 1 y 2, la LSTM promedia 87,34 ± 1,46 % (entre 85,65 % y 88,27 %) y la GRU 88,65 ± 0,08 %, con una desviación estándar unas 18 veces menor.

En la Fig. 1a, la GRU reduce la pérdida más rápido. Su mejor época fue la 3 o la 4, y después la pérdida de validación sube mientras la de entrenamiento sigue bajando. La LSTM mejora más despacio y es menos estable: con la semilla 2 su pérdida de validación salta a 0,548 en la época 3, y esa corrida termina con la peor exactitud. En la corrida principal, el modelo recupera el 90,2 % de las reseñas positivas y el 83,1 % de las negativas; la Fig. 1b muestra 502 negativas clasificadas como positivas frente a 294 errores en el sentido contrario.

Tabla 1. Métricas en el conjunto de prueba (5959 reseñas). LSTM s42 es la corrida principal; para las semillas 0, 1 y 2 se muestra el promedio ± la desviación estándar muestral.

Métrica	LSTM s42	LSTM (3 sem.)	GRU (3 sem.)
Exactitud	0,8664	0,8734 ± 0,0146	0,8865 ± 0,0008
F1 macro	0,8662	0,8733 ± 0,0146	0,8864 ± 0,0007
Precision negativa	0,8937	0,8855 ± 0,0257	0,8958 ± 0,0172
Recall negativa	0,8311	0,8576 ± 0,0108	0,8748 ± 0,0199
Precision positiva	0,8428	0,8625 ± 0,0095	0,8785 ± 0,0145
Recall positiva	0,9015	0,8890 ± 0,0283	0,8982 ± 0,0209
Pérdida val. (mejor)	0,3437	0,3063 ± 0,0268	0,2664 ± 0,0093
Pérdida prueba	0,3369	0,3119 ± 0,0246	0,2813 ± 0,0098
Pred. positivas (%)	53,6	51,7 ± 1,6	51,3 ± 2,0
Mejor época	5	4,7 ± 0,6	3,7 ± 0,6

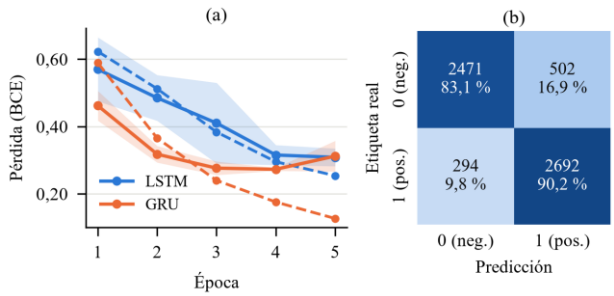


Fig. 1. (a) Pérdida de entropía cruzada binaria por época, promedio de las tres semillas de cada arquitectura, en entrenamiento (línea discontinua) y validación (línea continua); la banda es ± 1 desviación estándar de la validación. (b) Matriz de confusión en prueba de la LSTM con semilla 42; cada celda muestra el número de reseñas y el porcentaje respecto a la clase real.

La Tabla 2 muestra la reseña de prueba más corta de cada combinación de etiqueta real y predicción. Los aciertos contienen palabras de opinión explícitas (*unwatchable*, *great job*). En los errores el sentimiento es indirecto: "Read the book, forget the movie!" es negativa sin usar palabras negativas, y la reseña positiva mal clasificada elogia la película y cierra quejándose de que no está disponible en video en Australia ("What a waste").

Tabla 2. Reseña de prueba más corta de cada combinación de etiqueta real y predicción (LSTM, semilla 42), con la probabilidad asignada a la clase positiva. Las dos últimas filas son errores; [...] indica texto omitido.

Reseña	Real	Pred.	P(pos.)
Unwatchable. You can't even make it past the first three minutes. And this is coming from a huge Adam Sandler fan!!1	0	0	0,320
As usual, Sean Connery does a great job. Lawrence Fishburn is good, but I have a hard time not seeing him as Ike Turner.	1	1	0,960
Read the book, forget the movie!	0	1	0,737
What an amazing film. With very little dialogue, the whole story is told with glances and body language. [...] What a waste.	1	0	0,174

4. Discusión

La GRU fue más exacta y mucho más estable que la LSTM con los mismos datos. Su capa recurrente es más pequeña y converge más rápido: en la época 2 su pérdida de entrenamiento promedio ya es 0,366, frente a 0,512 en la LSTM. Ese ritmo tiene un costo, porque desde la época 3 o 4 la GRU sobreajusta: la pérdida de entrenamiento sigue bajando mientras la de validación sube. Es el compromiso entre sesgo y varianza [9], y restaurar los pesos de la mejor época de validación evitó que ese sobreajuste llegara a prueba. La LSTM todavía mejoraba en la época 5 en dos de las tres semillas, así que 5 épocas no le alcanzan para converger.

El rango de la LSTM muestra que su resultado depende de la inicialización y del orden de los lotes; la corrida principal (86,64 %) cae dentro de ese rango. Su sesgo hacia la clase positiva (53,6 % de predicciones positivas) tampoco es sistemático: con las otras semillas la proporción quedó entre 49,2 % y 53,3 %. En ambos modelos más del 97 % de los parámetros está en el embedding, entrenado desde cero con 27 806 reseñas, y las palabras poco frecuentes reciben vectores ajustados a las pocas reseñas en que aparecen.

Los errores de la Tabla 2 podrían deberse a que el modelo resume la reseña con el estado oculto del último token, que en una red unidireccional está más influido por lo leído al final. La reseña positiva que cierra con "What a waste" recibe una probabilidad positiva de 0,174, y en el acierto negativo el elogio final a Adam Sandler la deja en 0,320. Son solo dos ejemplos, así que es una hipótesis por comprobar.

La GRU queda cerca del 88,89 % de Maas *et al.* [2], pero ellos separan entrenamiento y prueba por película y nuestra división es por reseña: reseñas de una misma película pueden quedar en ambos conjuntos y hacer que el resultado sea algo optimista. Además, tres semillas dan una estimación gruesa de la variabilidad, el umbral de 0,5 no se ajustó y la deduplicación solo detecta textos idénticos. Proponemos quedarnos con la GRU, añadir *dropout* sobre el embedding e inicializarlo con GloVe [6], dar más épocas a la LSTM y probar una red bidireccional, que también lee la reseña de atrás hacia adelante.

Referencias

[1] B. Pang, L. Lee, and S. Vaithyanathan, "Thumbs up? Sentiment classification using machine learning techniques," in *Proc. Conf. Empirical Methods Natural Lang. Process. (EMNLP)*, 2002, pp. 79–86.

[2] A. L. Maas, R. E. Daly, P. T. Pham, D. Huang, A. Y. Ng, and C. Potts, "Learning word vectors for sentiment analysis," in *Proc. 49th Annu. Meeting Assoc. Comput. Linguistics: Human Lang. Technol. (ACL-HLT)*, 2011, pp. 142–150.

[3] Y. Bengio, P. Simard, and P. Frasconi, "Learning long-term dependencies with gradient descent is difficult," *IEEE Trans. Neural Netw.*, vol. 5, no. 2, pp. 157–166, 1994.

[4] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural Comput.*, vol. 9, no. 8, pp. 1735–1780, 1997.

[5] M. Yasser H., "IMDB movie ratings sentiment analysis," Kaggle, 2022. [Online]. Available: https://www.kaggle.com/datasets/yasserh/imdb-movie-ratings-sentiment-analysis

[6] J. Pennington, R. Socher, and C. D. Manning, "GloVe: Global vectors for word representation," in *Proc. Conf. Empirical Methods Natural Lang. Process. (EMNLP)*, 2014, pp. 1532–1543.

[7] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2015.

[8] K. Cho *et al.*, "Learning phrase representations using RNN encoder-decoder for statistical machine translation," in *Proc. Conf. Empirical Methods Natural Lang. Process. (EMNLP)*, 2014, pp. 1724–1734.

[9] I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*. Cambridge, MA, USA: MIT Press, 2016.